SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS: 4 **Total Marks:50**

Annexure A

CLASS TEST

Code of Conduct:

I ____S. Pooja (192525211)______________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Pooja

Q. No	Understanding of Concepts (25)	Explanation (25)	Application (20)	Organization (15)	Accuracy(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 30**	**/ 20**	**/ 15**	**/ 10**	**/ 100**

Annexure A

CLASS TEST

Q1. Construct three address code for the following

if((a < b) and ((c < d) or (a > d))) then

z=x + y *z

else

z = z + 1

Q2.Translate (a < b) and (c < d) or (e < f) into three address statements using back patching.

Q3. Construct three address code using the following production for Boolean expression (a > b) or (c < d) or (e < f)

B → B1 || B2

B → B1 && B2

B → ! B1

B → E1 relop E2

B → true

B → false

Q4. Generate the three address code for the following program fragment-

while (A < C and B > D) do

if A = 1 then C = C + 1

else while A < = D do

A = A + B

Date: 18.08.26

CO4 - AT1
Class TEST

Name: POOJA.S
Regno: 192524211
Code: CSA1402
Course: Compiler Design

1. Construct three address code for the following
if ((a<b) and ((c<d) or (a>d))) then z = x+y*z
else
z = z+1

38/40

Given Statement:
if ((a<b) and ((c<d) or (a>d))) then
z = z+1

Step 1: understand the boolean conditions

(a<b) AND ((c<d))

a<b
c<d
a>d

The structure is (a<b) AND [(c<d) OR (a>d)]

Step 2: Create Tables
Use Labels for the conditions checks, then part, else part, and the end of the statement
L1, L2, L3, L4, L5, L6.

Step 3: Generate TAC for the condition

L1:
if a<b goto L2
goto L5

L2:
if c<d goto L2
goto L5

L3:
if a>d goto L4
goto L3

L3:
if a>d goto L4
goto L5

If a<b is false, the complete AND expression is false, so control goes to the else part. If c<d is true, expression is evaluated. If a<b is true, control goes to the then part
or a>d is true, control goes to the then part

Step 4: Generate TAC for the then part. The statement is $z=x+y*z$. Multiplication has higher precedence than addition.

L4:
$t1 = y*z$
$t2 = x+t1$
$z = t2$
goto L6

Step 5: Generate TAC for the ELSE part

L5:
$t3 = z+1$
$z = t3$

Final TAC

L1:
if a<b goto L2
goto L5

L2:
if c<d goto L4
goto L3

L3:
if a>d goto L4
goto L5

L4:
$t1 = y*z$
$t2 = x+t1$
$z = t2$
goto L6

L5:

$t3 = z+1$

$z = t3$

L6:

---

2. Translate (a<b) and (c<d) or (e<f) into three address statements using back patching

Given (a<b) and (c<d) or (e<f)

Step 1: Apply operator precedence

AND has higher precedence than OR, so the expression is interpreted as:

[(a<b) AND (c<d)] or (e<f)

Step 2: Generate incomplete jumps

1. if a<b goto _
2. goto _
3. if c<d goto _
4. goto _
5. if e<f goto _
6. goto _

The underscore represents a target address that is not known yet

Step 3: Handle AND

For (a<b) AND (c<d), if a<b is true, control must continue to the second condition

Backpatch state...

1. if a<b goto 3

Step 4: Handle OR

if the first AND expression is false, the second OR operand e<f must be evaluated.

Backpatch statement 2 to statement 5

Backpatch statement 4 to statement 5

Step 5: Final backpatched code

1. if a<b goto 3
2. goto 5
3. if c<d goto 7
4. goto 5
5. if e<f goto 7
6. goto 8
7. TRUE
8. FALSE

Backpatching means generating jump instructions 1st and following target address after the required labels or target become known

Construct three address code using the follow production for Boolean expression (a>b) or (c<d) or (e<f)

B -> B1||B2   B -> true

B -> B1&&B2   B -> false

B -> !B1

B -> E1 relop E2

L(c<d)

Conditio Step 1: Understand the expression

of

[(a>b) OR (c<d)] OR

Because this is an OR expression, if any one of the three relational condition is true, the complete Boolean expression is true

Step 2:

L2:
if (a >b) goto L TRUE

goto L2

Step 3: Check the second condition

L2:
if c <d goto LTRUE

goto L3

Step 4: check the third condition

L3:
if e<f goto LTRUE
if e<f goto LTRUE

goto LFALSE

FINAL TAC for Q3

L1:
if a >b goto LTRUE

goto L2

L2:
if c<d goto LTRUE

goto L3

LB:
if e<f goto LTRUE
goto LFALSE

LTRUE:
B→TRUE
goto LEND

LFALSE:
B→FALSE

LEND:

4. Generate the three address code for the following program fragment - while (A<C and B>D)
do if A=1 then C=C+1
else while A<=D do
A=A+B

Complete TAC

L1:
if A<C goto L2
goto LEND

L2:
If B>D goto L3
goto LEND

L3:
If A==1 goto L4
goto L5

L4:
t1=C+1
C=t1
goto L1

L5:
if A<=D goto L6
goto L1

L6:
t2=A+B
A=t2
goto L5

LEND:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	15%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps